Foundations of Transformer Models

John Snyder

STAT 9340: Spring 2023

Attention Is All You Need

In Attention Is All You Need(2017), the authors introduced the **Transformer** architecture, a novel approach to sequence-to-sequence tasks that relies solely on attention mechanisms, overcoming the limitations of traditional RNNs and CNNs. Key contributions include

1. **The Transformer architecture**: A new encoder-decoder neural network architecture that relies entirely on self-attention mechanisms, dispensing with the need for recurrent or convolutional layers.
2. **Multi-head self-attention**: A crucial component of the Transformer, multi-head self-attention allows the model to capture different aspects of the input, improving its representation and facilitating parallelization.
3. **Positional encoding**: To address the lack of inherent order information in the attention mechanism, the authors introduced positional encoding, which injects information about the position of each word in the sequence.

Attention Is All You Need(cont)

This architecture achieved immediate state-of-the-art results on language translation tasks, and has since become the primary building block for most NLP tasks.

- Text Classification
- Named Entity Recognition (NER)
- Question Answering
- Masked Language Modeling
- Text Generation

Transformers are excellent at modeling all kinds of non NLP related data as well, including

- Speech Recognition
- Protein Folding
- Vision Transformers for imaging tasks
- Time Series Forecasting(TFT)

How we build to Transformers

Transformers are building blocks whose applications build through the following concepts

- Encoder-decoder based transformer models
  - We have seen the general idea of this already
- Self-attention mechanisms
  - Will introduce some special matrices named **query**, **key** and **value**
- Positional encoding
  - Using sinusoidal functions
- Position-wise feed-forward networks

These allow for inputs in a sequence to be processed in parallel, rather than sequentially as was the case with recurrent models.

Encoder-Decoder Structure

Many Transformer models are applications of the Encoder-Decoder approach.

- **Encoder**: Processes the input and outputs a representation made up of real numbers
  - N identical layers, each containing a multi-head self-attention mechanism and a position-wise feed-forward network
- **Decoder**: Generates the output sequence from the output of the encoder
  - N identical layers, each containing a masked multi-head self-attention mechanism, a multi-head self-attention mechanism (with encoder outputs), and a position-wise feed-forward network

In each component of these models, we have repeated applications of a similar set of calculations.

Quick review of *embedding* in text processing.

The process of *tokenizing* breaks the input into chunks (words in this case). The population of words forms a vocabulary which allows us to transform tokens into integers.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Embedding
text = ['This is one cool sentence about word embeddings.',
        'The weather today is very cool, ill do some NLP.']

tokenizer = Tokenizer()
tokenizer.fit_on_texts(text)
sequences = tokenizer.texts_to_sequences(text)
print(sequences)
```

```
## [[3, 1, 4, 2, 5, 6, 7, 8], [9, 10, 11, 1, 12, 2, 13, 14, 15, 16]]
```

Quick review of *embedding* in text processing.

Once we have a numeric representation of tokens, we need to make sure everyone is the same length

```
max_length = max([len(seq) for seq in sequences])
padded_sequences = pad_sequences(sequences, maxlen=max_length,
                                 padding='post')
print(padded_sequences)
```

```
## [[ 3  1  4  2  5  6  7  8  0  0]
##  [ 9 10 11  1 12  2 13 14 15 16]]
```

```
embedding_dim = 5 # Dimension we would like to represent each word
embedding_layer = Embedding(input_dim=len(tokenizer.word_index)+1,
                            output_dim=embedding_dim)
# Embedding TF layer to get n x d representation of each word
encoded_sequences = embedding_layer(padded_sequences)
encoded_sequences.shape
```

```
## TensorShape([2, 10, 5])
```

Building the Query, Key, and Value Matrices

Building towards self attention begins with the *Query*, *Key*, and *Value* matrices.

Say we have n *embedded* words in a sentence such that each is a d-dimensional vector $x_i \in \mathbb{R}^d$ and the sentence $X = [x_1, \ldots, x_n]^T \in \mathbb{R}^{n \times d}$ is an $n \times d$ matrix.

- This representation does not inherently consider the surroundings of any given word.

For a sentence with n words, self-attention computes n self-attention representations $A_1, \ldots, A_n$ for the n words.

For each embedded word $x_i \in \mathbb{R}^d$ we compute a query $q_i \in \mathbb{R}^d$, key $k_i \in \mathbb{R}^d$ value $v_i \in \mathbb{R}^d$, vectors, which make up the rows of X, Q, K, and V by:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V,$$

where W^Q, W^K, and W^V are learned weight matrices.

Attention Generally

When writing or translating text we typically work with different parts of the text at different times, going back to other parts as needed. This process is not simply sequential like we discussed using models we had previously. Attention helps solve this problem.

A simple form of attention is applied to regression. Given samples x_i, y_i of a function, we can estimate the value $y = f(x)$ by weighting each y_i as a function of $\alpha(x, x_i)$, which decreases the farther apart x and x_i are. For example, we can estimate:

$$y = f(x) = \sum_i \alpha(x, x_i)\, y_i$$

where in attention x is called the query, x_i is called a key, and y_i is a value. We can think of the query as a "user input" which seek the output y. The keys are other potential sources of information which have their *own* outputs.

Self-Attention Mechanisms

The self-attention mechanism of transformers allows the model to weigh the importance of different words in the input sequence when generating the output sequence.

- **Scaled Dot-Product Attention**:

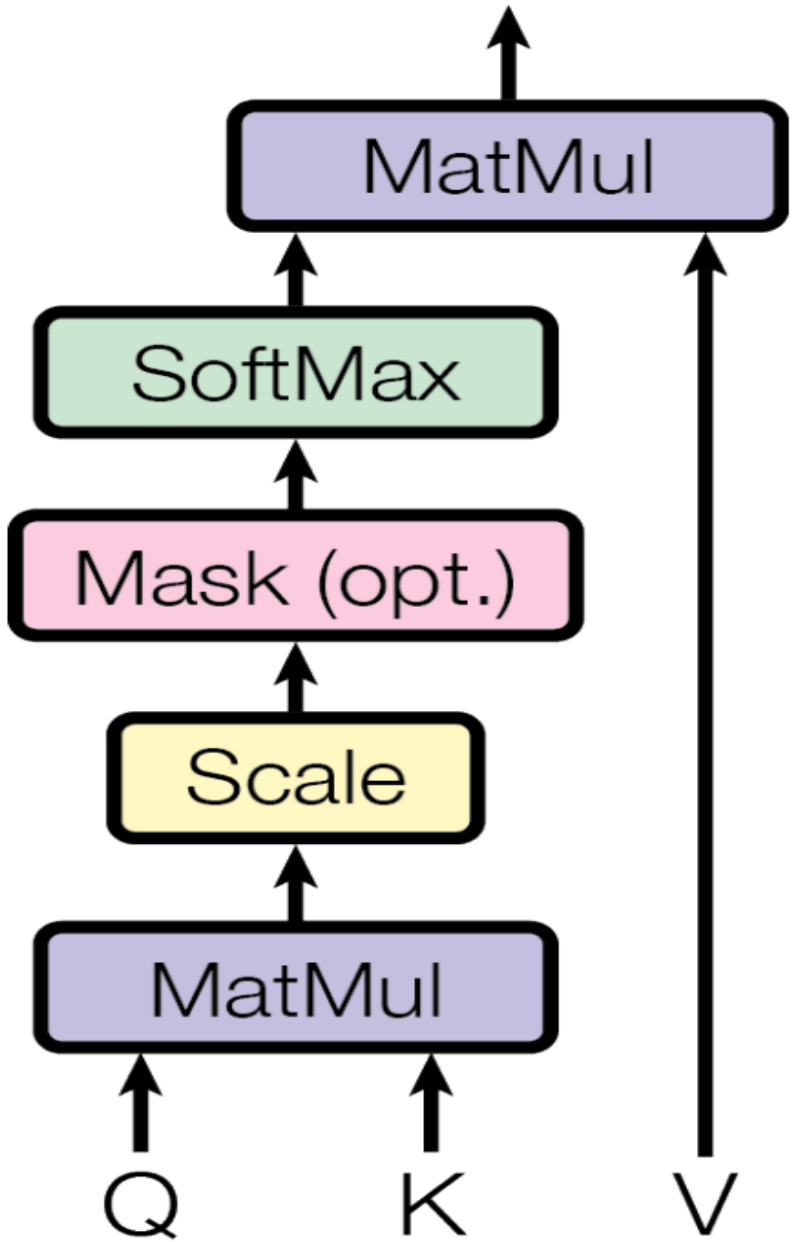
$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, and V are query, key, and value matrices, respectively, and d_k is the dimension of the key vectors. Here, the softmax function is applied *row-wise*

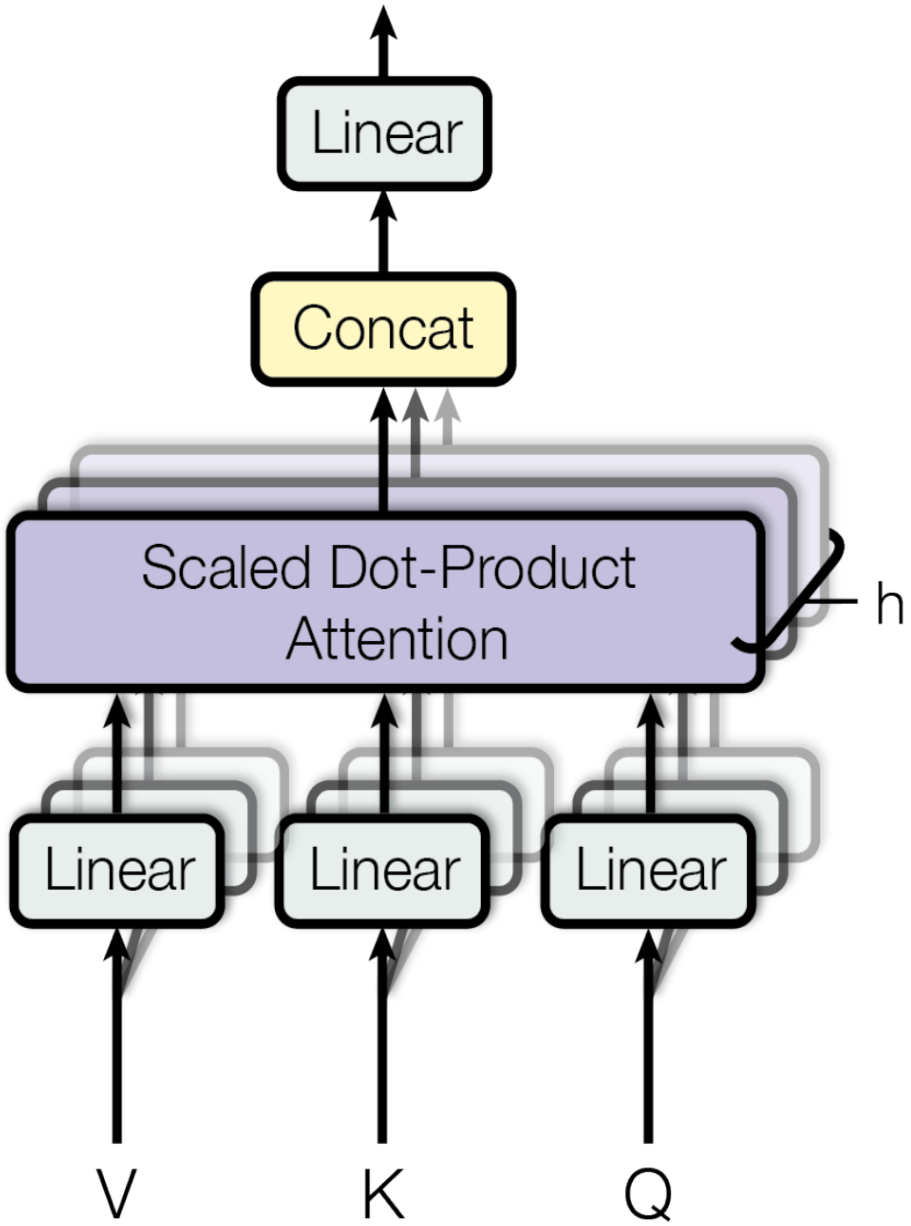
- **Multi-Head Attention**:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}\left(\text{head}_1, \ldots, \text{head}_\text{h}\right) W^O \\ \text{where head} &= \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right) \end{aligned}$$

Attention Diagram



Multi-Head Attention Diagram



Discessting self attention

Let's look at what the proposed attention gives us for the first embedded word x_1, with its rows of the query, key, and value matrices q_1, k_1, and v_1.

First note that the first row of QK^T is given by

$$[q_1 \cdot k_1, \quad q_1 \cdot k_2, \quad q_1 \cdot k_3, \ldots q_1 \cdot k_n]$$

So we have the collection of dot products between the query vector of x_1 and the key vector of every other word in the sequence, including itself.

This row represents in a sense the raw attention scores of a word with respect to all words in the sequence because elements will be high if $q_i \approx k_j$.

The scaling by d_k and softmax function then normalize these values.

- Note that the scaling is to help avoid vanishing gradients, as the unnormalized attention scores can be quite large in magnitude.

Discessting self attention

Once we have applied the softmax function, the rows of the resulting $n \times n$ matrix represent attention weights indicating the relevance of other words to the current word. That is, the more influence the information from one word has on the context-aware representation of the other word, the larger the attention.

Multiplying by V gives the final attention scores.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Each row of the attention matrix represents a *context-aware* representation of a word in the input sequence. The $(n \times d)$ value matrix is just another representation of the inputs, so dotting this with the normalized attention creates a result that will tell the model which parts to "focus on" during the training task.

Self-Attention Matrix Example

Self-attention

Probability score matrix

	Hello	I	love	you
Hello	0.8	0.1	0.05	0.05
I	0.1	0.6	0.2	0.1
love	0.05	0.2	0.65	0.1
you	0.2	0.1	0.1	0.6

Context Aware representations

Context aware representation just means we took the raw input and captured structural dependence among all inputs, and projected this information back onto the data point.

For example the sentence "After I went to the bank and saw my balance I went to a river bank to balance my emotions" has the same words with vastly different meanings. In this example, "balance" and "bank" would have vastly different context aware representations at their different positions.

This method of capturing dependence can also be applied to imaging problems, time series, really anywhere that dependent data lives.

Incorporating positional information

If we are feeding embedded words into a sequential model like an LSTM for modeling, positional information has already been incorporated by virtue of how LSTM layers process data.

- Advantage: understands the ordering
- Disadvantage: Slow

With what has been discussed regarding self attention thus far, we can see that the entire self attention matrix can process each word in parallel. Thus, no positional information is included.

- Advantage: Fast
- Disadvantage: No understanding of ordering

To fix this problem, Vaswani et al. (2017) proposed a method of injecting positional information *into* the embedded representations of the words by sweeping through different frequencies of sin and cos functions.

Positional Encoding

To incorporate positional information into the input, we will use the positional encoding scheme. The $n \times d$ matrix P is created by

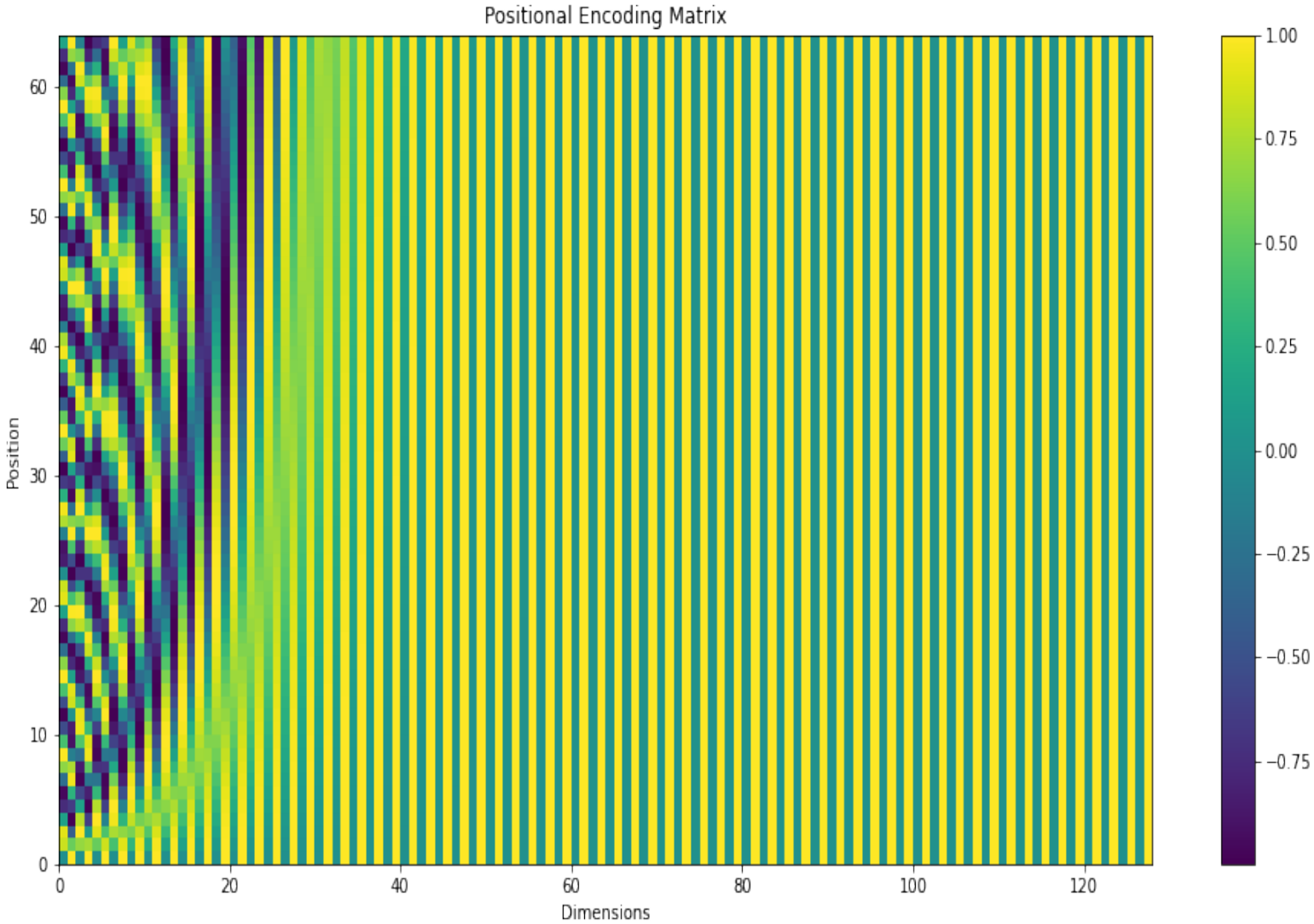
$$P_{(p,i)} = \begin{cases} \sin\left(\frac{p}{10000^{\frac{2i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{p}{10000^{\frac{2i-1}{d}}}\right) & \text{if } i \text{ is odd} \end{cases} \tag{1}$$

where:

- p is the position of the word in the input sequence
- i is the dimension of the encoding
- d is the total number of dimensions for the encoding

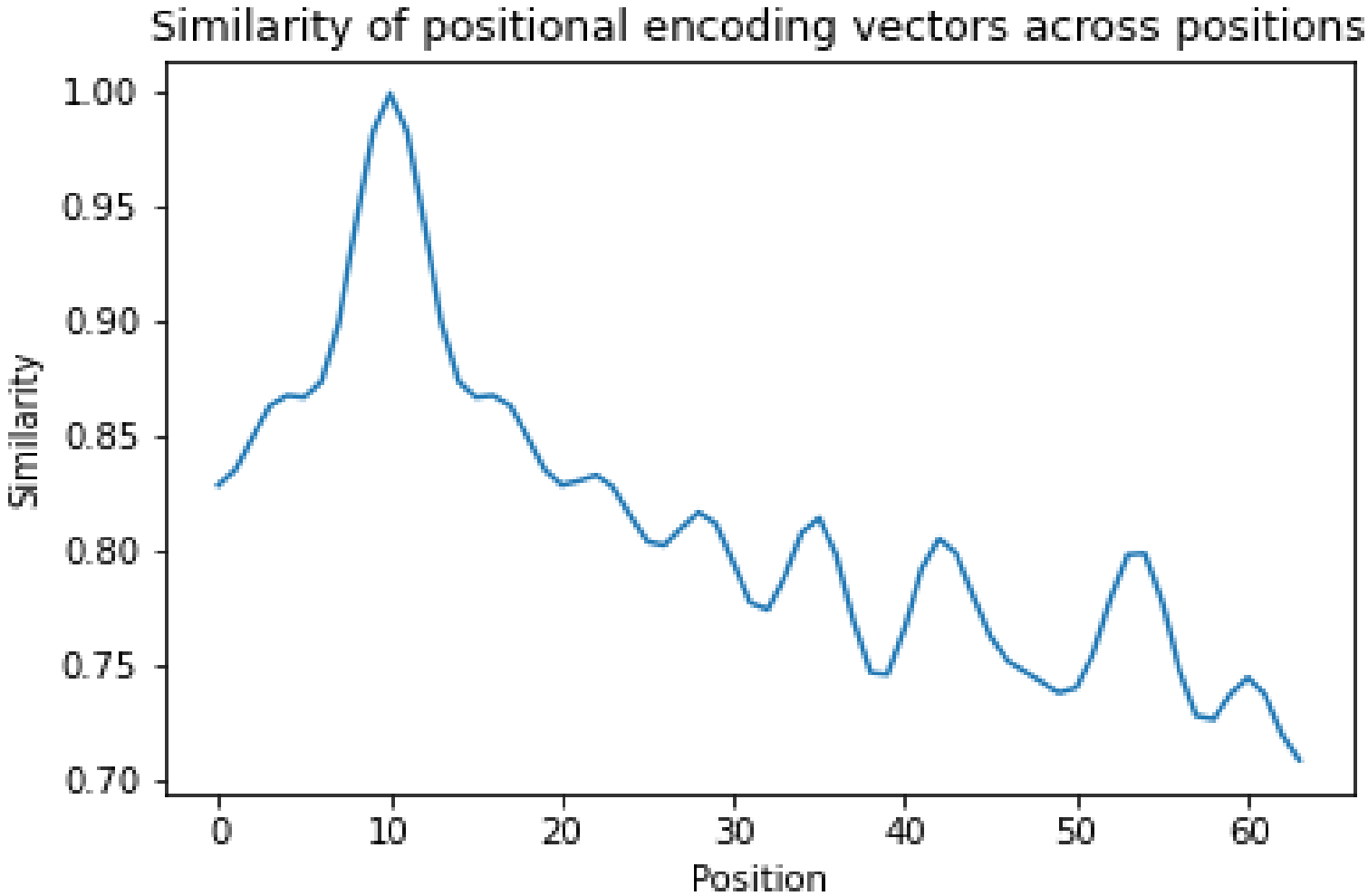
These values are then added to the embedded X matrix.

Positional Encoding Matrix



Similarity of Positional Encoding Vectors

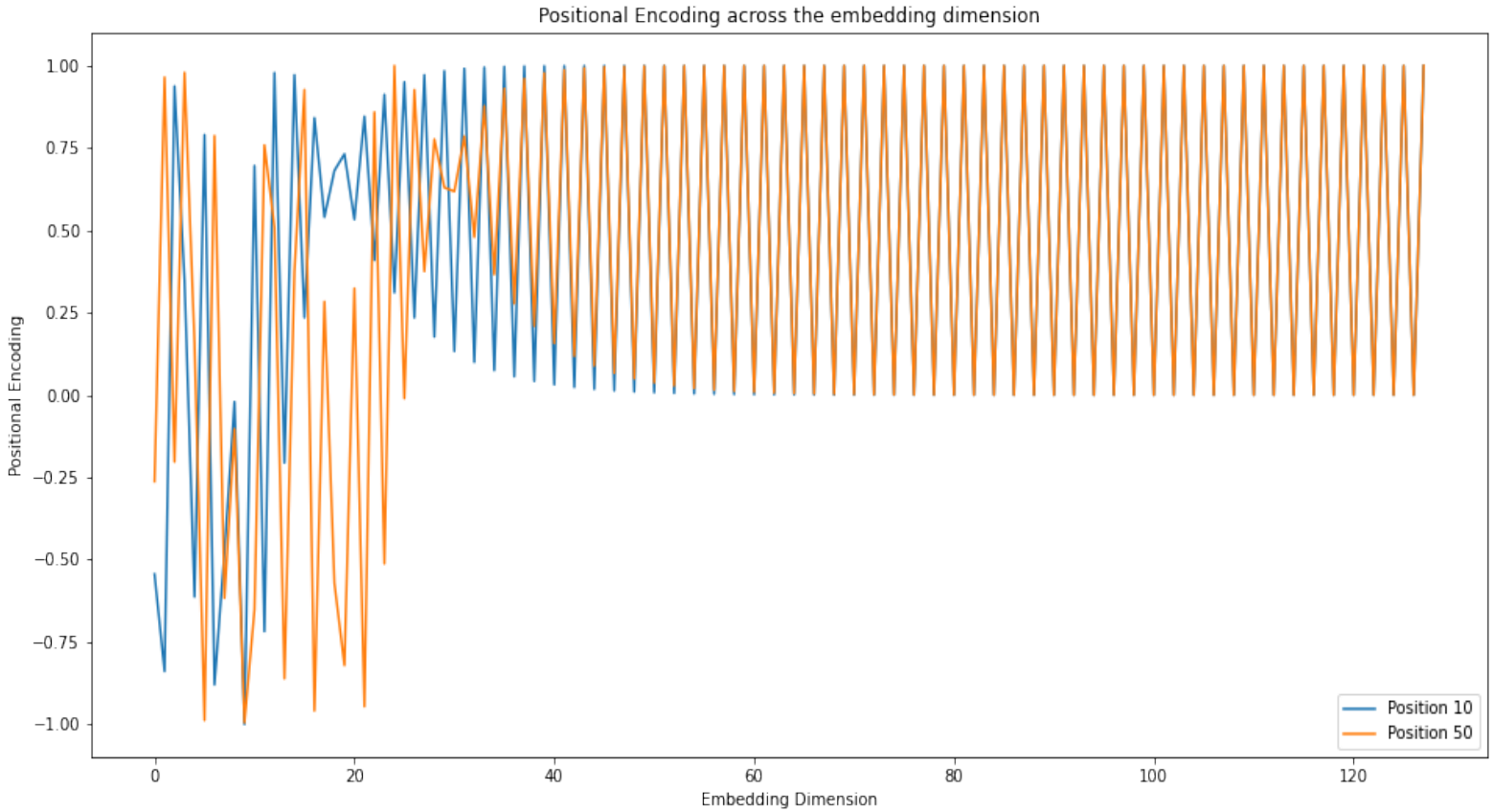
Norming each position and dotting with position 10 to examine similarity we get:



Convergence across the encoding dimension

As the dimension of the encoding approach the total dimensions, the value of the positional encoding does not change as drastically with position.

$$\frac{p}{10000^{\frac{2i}{d}}} \approx \frac{p}{10000^2}$$



Depending on the task at hand, the model will learn to attend to whichever portion of the embedding space solves the problem.

Masking

Consider again our sentence of n embedded words $x_1, x_2, \ldots, x_n$ and the embedded $n \times d$ matrix $X = [x_1, x_2, \ldots, x_n]^T$. The raw attention scores A were calculated as:

$$A = \frac{QK^T}{\sqrt{d_k}}$$

We now create a mask M of the same shape as A. For each position i in the sequence, we mask the future positions by setting the corresponding mask value to $-\infty$ (in practice, a large negative number):

$$M_{ij} = \begin{cases} 0, & \text{if } j \leq i \\ -\infty, & \text{if } j > i \end{cases}$$

The masked attention scores are computed by adding the mask M element-wise to the original attention scores A:

$$\alpha_{\text{masked}} = \text{softmax}\left(A_{\text{masked}}\right) = \text{softmax}\left(A + M\right)$$

We then MatMul by V as typical.

Masking example

Take the sentence "I love my cat." Masking would essentially process the input as:

```
[I,     -1e11,  -1e11,  -1e11],
[I,      love,  -1e11,  -1e11],
[I,      love,     my,  -1e11],
[I,      love,     my,    cat]
```

When the model processes:

- "I": it can only attend to itself,
- "love": it can only attend to "I" and "love"
- "cat": it can attend to all the words in the sequence, including "I", because it's the last token

So we have an attention score $A_{\text{I,cat}}$, but this is only derived when the entire sequence is available.

Masking summary

The masking process is critical for maintaining the autoregressive properties of what the self attention blocks learn.

- Language modeling
- Machine translation
- Text summarization
- Dialogue systems
- Autoregressive sequence generation tasks for non text systems
  - Speech/music generation
  - image captioning
  - etc

The success of GPT models, such as GPT-3, can be partly attributed to their ability to effectively model the causal structure in the input data, enabled by the proper application of lookahead masks in the self-attention mechanism.

Two Final Components

There are two additional key aspects of the transformer blocks used in the encoders

- *Residual connections* are used to directly connect the input to the block to the output of self attention, fusing them both together
  - This allows the gradients access to both self attention as well as the input itself during back propagation.

- *Position-wise feed-forward neural network(FFNN)* : After self attention, the outputs are passed through two dense layers with weights are shared across positions.

Complete Encoder Architecture

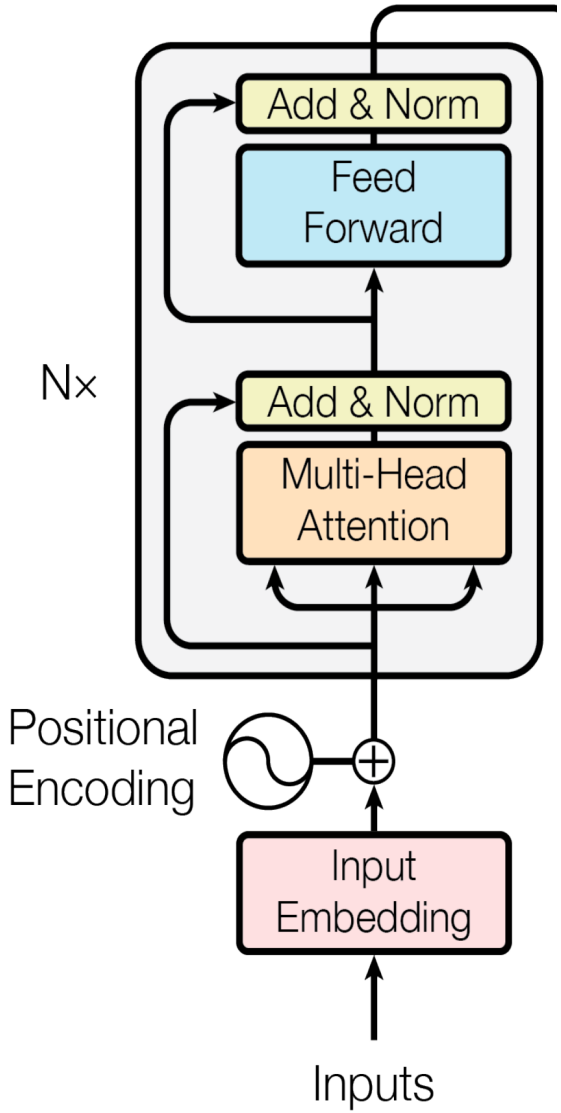
In summary the encoder portion of a transformer based model has the following steps.

1. Input Embedding
2. Positional Encoding

Then, for **each** of N transformer blocks, we conduct the following steps.

1. Multi-head Self-Attention
2. Add & Norm
3. Feed-Forward Neural Network
4. Add & Norm
   - residual connection from normalized self attention output

The output from the final Add & Norm layer in one Transformer block serves as the input for the next Transformer block in the stack.



Tensorflow code to illustrate transformer block

```
import tensorflow as tf
from tensorflow.keras.layers import (LayerNormalization, MultiHeadAttention,
                                     Dense, Dropout, Add)

def transformer_encoder_layer(input_layer, num_heads, d_ff, dropout_rate):
    # Two positional arguments to MultiHeadAttention: (query, key)
    # value is assumed from key
    attn = MultiHeadAttention(num_heads=num_heads,
                              key_dim=input_layer.shape[-1]
                             )(input_layer, input_layer)
    attn = Dropout(dropout_rate)(attn)

    # Bring in residual connection for 1st Add&Norm layer
    norm1 = LayerNormalization(epsilon=1e-6)(Add()([input_layer, attn]))

    # FFNN: Remember to get back to original shape!
    ffnn = Dense(d_ff, activation='relu')(norm1)
    ffnn = Dense(input_layer.shape[-1])(ffnn)
    ffnn = Dropout(dropout_rate)(ffnn)

    # Residual connection from normalized self attention for 2nd Add&Norm
    norm2 = LayerNormalization(epsilon=1e-6)(Add()([norm1, ffnn]))

    return norm2
```

Example usage of the above to build an encoder

```
def transformer_encoder(input_layer, num_layers,
                        num_heads, d_ff, dropout_rate):
    x = input_layer
    for _ in range(num_layers):
        x = transformer_encoder_layer(x, num_heads, d_ff, dropout_rate)
    return x

sequence_length = 50
embedding_dim = 512
num_layers = 6
num_heads = 8
d_ff = 2048
dropout_rate = 0.1

input_layer = tf.keras.Input(shape=(sequence_length, embedding_dim))
encoder_output = transformer_encoder(input_layer, num_layers,
                                     num_heads, d_ff, dropout_rate)

encoder_model = tf.keras.Model(inputs=input_layer, outputs=encoder_output)
encoder_model.summary()
```

Encoder summary: lots of weights!

```
 Layer (type)                Output Shape              Param #     Connected to
==================================================================================================
 input_1 (InputLayer)        [(None, 50, 512)]         0           []

 multi_head_attention (MultiHea  (None, 50, 512)       8401408     ['input_1[0][0]',
 dAttention)                                                        'input_1[0][0]']

 dropout (Dropout)           (None, 50, 512)           0           ['multi_head_attention[0][0]']

 add (Add)                   (None, 50, 512)           0           ['input_1[0][0]',
                                                                    'dropout[0][0]']

 layer_normalization (LayerNorm  (None, 50, 512)       1024        ['add[0][0]']
 alization)

 dense (Dense)               (None, 50, 2048)          1050624     ['layer_normalization[0][0]']

 dense_1 (Dense)             (None, 50, 512)           1049088     ['dense[0][0]']

 dropout_1 (Dropout)         (None, 50, 512)           0           ['dense_1[0][0]']

 add_1 (Add)                 (None, 50, 512)           0           ['layer_normalization[0][0]',
                                                                    'dropout_1[0][0]']
...
Total params: 63,019,008
Trainable params: 63,019,008
Non-trainable params: 0
__________________________________________________________________________________________________
```

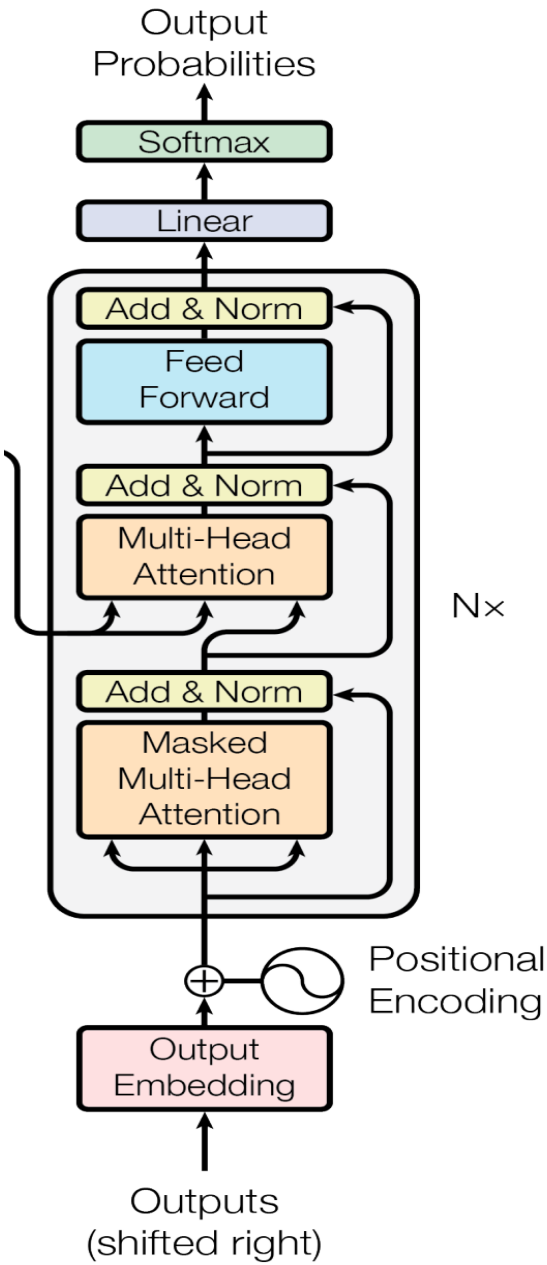
Decoder architecture

The structure of a transformer block in the decoder is similar to the encoder, with one key difference.

- There are *two* (potentially multi head) self attention mechanisms.

1. The first uses *masked* self attention since its purpose is to ultimately to predict outputs in a "sequential" fashion.
2. The second takes input from two sources
   - The output of the encoder
   - The output of the maskef self attention block

The second self attention block is called an "Encoder-decoder attention mechanism," "cross-attention," or "cross-modal attention." The masking process is applied in the attention matrix, so the calculations can still be conducted in parallel!



Complete Encoder-Decoder architecture

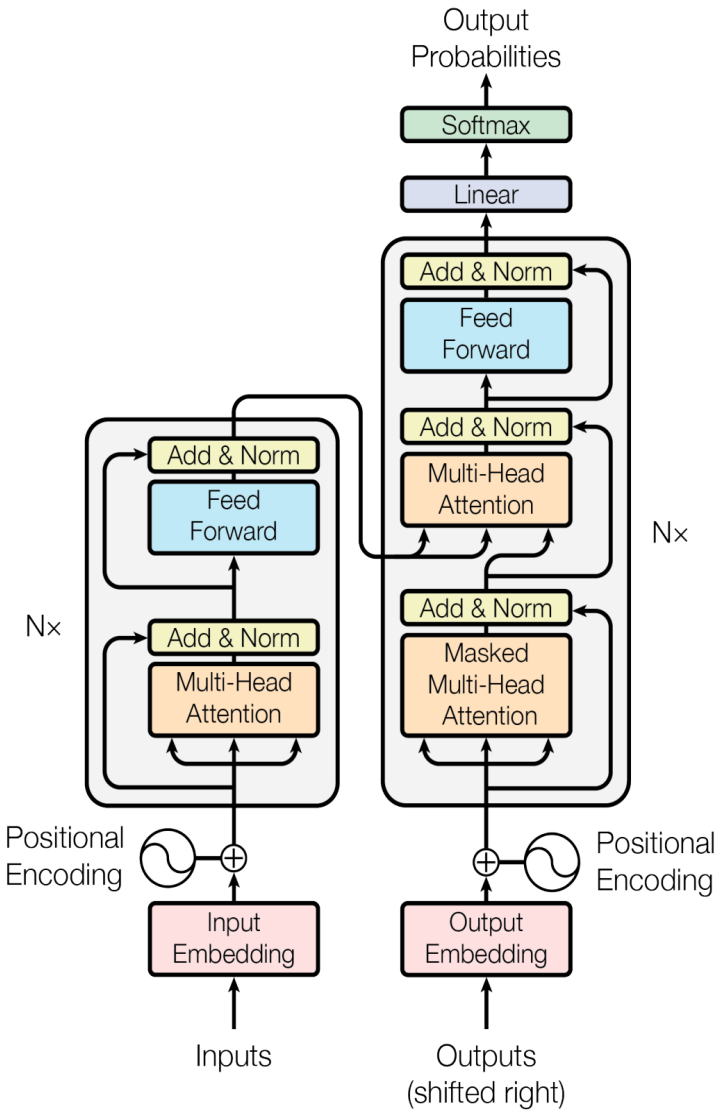
Say we wish to translate the sentence "I love my cat." from English to German. Data in the model would be processed as:

Input:
`["I", "love", "my", "cat", "."]`

Output:
`["Ich", "liebe", "meine", "Katze", "."]`

Output(shifted right):
`["<start>", "Ich", "liebe", "meine", "Katze", "."]`

During training, the model will have access to pairs of inputs and outputs. It predict **entire** target sequences at one time, using the masked attention to enforce the autoregressive nature of this process.



Prediction

During inference in a sequence-to-sequence task:

- The encoder processes the **entire** input sequence and produces a representation of the input.
- The decoder then starts with the token and generates the output sequence token by token in an autoregressive manner.
- The predicted first token is then used as input for the decoder (along with the token), and the decoder predicts the second token in the target language based on the encoder's output and the previous decoder tokens.

This process continues until the decoder generates the entire output sequence or reaches a predefined maximum length or an end-of-sequence token.

`["<start>" -> "Ich" -> "liebe" -> "meine" -> "Katze" -> "."]`

Note that this process does not happen in parallel.

Attention computational complexity

Attention is great at capturing highly complex relationships between inputs, but has some computational considerations.

- The $n \times n$ self-attention matrix requires n^2 multiplications and additions for every input in our sentence.
  - This means that this process has $O(n^2)$
- The quadratic complexity of the self attention process can be a major computational issue when working with long sequences.

Rethinking Attention with Performers(2022) provides an efficient approximation to the self attention process which can be computed with complexity $O(n)$.

Rough sketch of performer

The elements of the attention matrix $A(i,j)$ are given by

$$\mathbf{A}(i,j) = \mathrm{K}\left(\mathbf{q}_i^\top, \mathbf{k}_j^\top\right)$$

where $\mathbf{q}_i/\mathbf{k}_j$ are the i^{th} /j^{th} rows in $\mathbf{Q}/\mathbf{K}$ and kernel $\mathrm{K} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_+$

- The kernel function $K(x,y)$ is used to approximate the similarity between the current pair of query and key vectors and is defined as

$$\mathrm{K}(\mathbf{x},\mathbf{y}) = \mathbb{E}\left[\phi(\mathbf{x})^\top \phi(\mathbf{y})\right]$$

where $\phi : \mathbb{R}^d \to \mathbb{R}_+^r$ is a random feature mapping function that maps the query and key vectors (x and y) into a (typically) higher dimensional space.

The kernel function $K(x,y)$ is a dot product which measures the similarity between the transformed query and key vectors. In essence, this is how we will approximate the softmax function.

The kernal trick

This approach is used in various algorithms to implicitly compute nonlinear functions in a transformed space without explicitly computing the transformation per se. We replace the dot product in the original space with a kernel function that represents it in the transformed space.

Recall the radial basis function (RBF) kernel in support vector machines. We often used this kernel to separate classes in 2d space when the boundary was highly nonlinear.

$$K(x, y) = \exp(-\gamma||x - y||^2) = \sum_{n=0}^{\infty}((-\gamma||x - y||^2)^n/n!)$$

The series expansion shows that the RBF kernel can be viewed as a dot product between the data points x and y in an infinite-dimensional feature space, where each dimension corresponds to a term in the series expansion.

Obviously, we do not need to use this infinite dimensional representation, we can just use the kernal directly. The reason it works, however, is *because of* the infinite expansion.

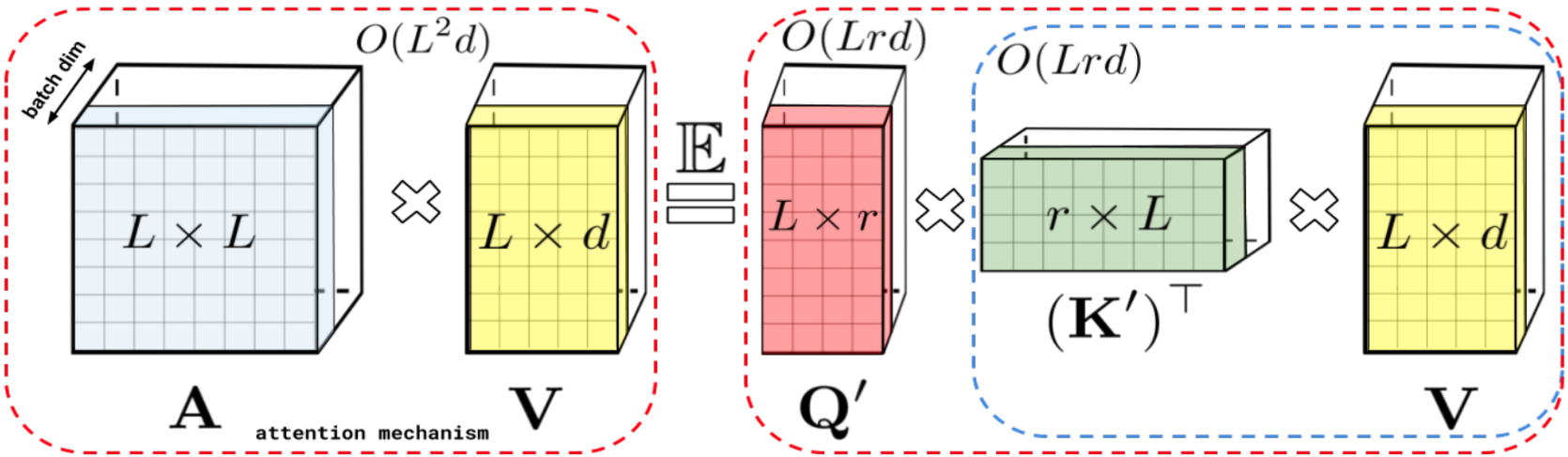
Approximated softmax

Once we have $\phi : \mathbb{R}^d \to \mathbb{R}^r_+$, define $\mathbf{Q}', \mathbf{K}' \in \mathbb{R}^{L \times r}$ with rows given as $\phi\left(\mathbf{q}_i^\top\right)^\top$ and $\phi\left(\mathbf{k}_i^\top\right)^\top$. Note that we will not explicitly compute these!

The kernel function K leads directly to the efficient attention mechanism of the form:

$$\widehat{\text{Att}}_{\leftrightarrow}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \hat{\mathbf{D}}^{-1}\left(\mathbf{Q}'\left((\mathbf{K}')^\top \mathbf{V}\right)\right), \quad \hat{\mathbf{D}} = \text{diag}\left(\mathbf{Q}'\left((\mathbf{K}')^\top \mathbf{1}_L\right)\right)$$

(note: paper uses L where we have used n)



The form of ϕ

The authors provide a specific form of ϕ to ensure we have an unbiased estimate of the softmax function in the attention mechanism.

$$\phi(\mathbf{x}) = \frac{h(\mathbf{x})}{\sqrt{m}} \left(f_1\left(\omega_1^\top \mathbf{x}\right), \ldots, f_1\left(\omega_m^\top \mathbf{x}\right), \ldots, f_l\left(\omega_1^\top \mathbf{x}\right), \ldots, f_l\left(\omega_m^\top \mathbf{x}\right) \right)$$

- x is the input (query or key vector)
- $h(x)$ is a non-negative function.
  - Ensures that the transformed feature space is non-negative
- m is the number of random features per base function.
- $\omega_1, \ldots, \omega_m$ are random vectors drawn from a suitable distribution (Gaussian).
- $f_1, \ldots, f_l$ are base functions applied element-wise to the input vector.

These values are then used to adjust the dimension of the original Q and K matrices. Taking random samples via ω_i in this way will yield a dot product which is roughly equal to the softmax'd self attention matrix in expected value.

Section 1

Some specific Transformer models

Overview

Transformers have revolutionized many modern applications

- BERT (Bidirectional Encoder Representations from Transformers): Developed by Google for NLP tasks
  - Useful for all kinds of tasks
- GPT (Generative Pre-trained Transformer): Developed by OpenAI for generating natural language text.
  - Input text, the model will continue predicting tokens
- T5 (Text-to-Text Transfer Transformer): Developed by Google that can perform a wide range of NLP tasks.
  - Text classification, translation, Q&A
- DALL-E: Generates images from text
- ViT: Vision transformers use attention to learn local features in images. Surpassed most cutting edge CNN models.
- TFT (temporal fusion transformers): Uses attention for time series forecasting.